



Departamento de Informática e Matemática Aplicada – DIMAp

Spring Config Server

Prof. Nélio Cacho

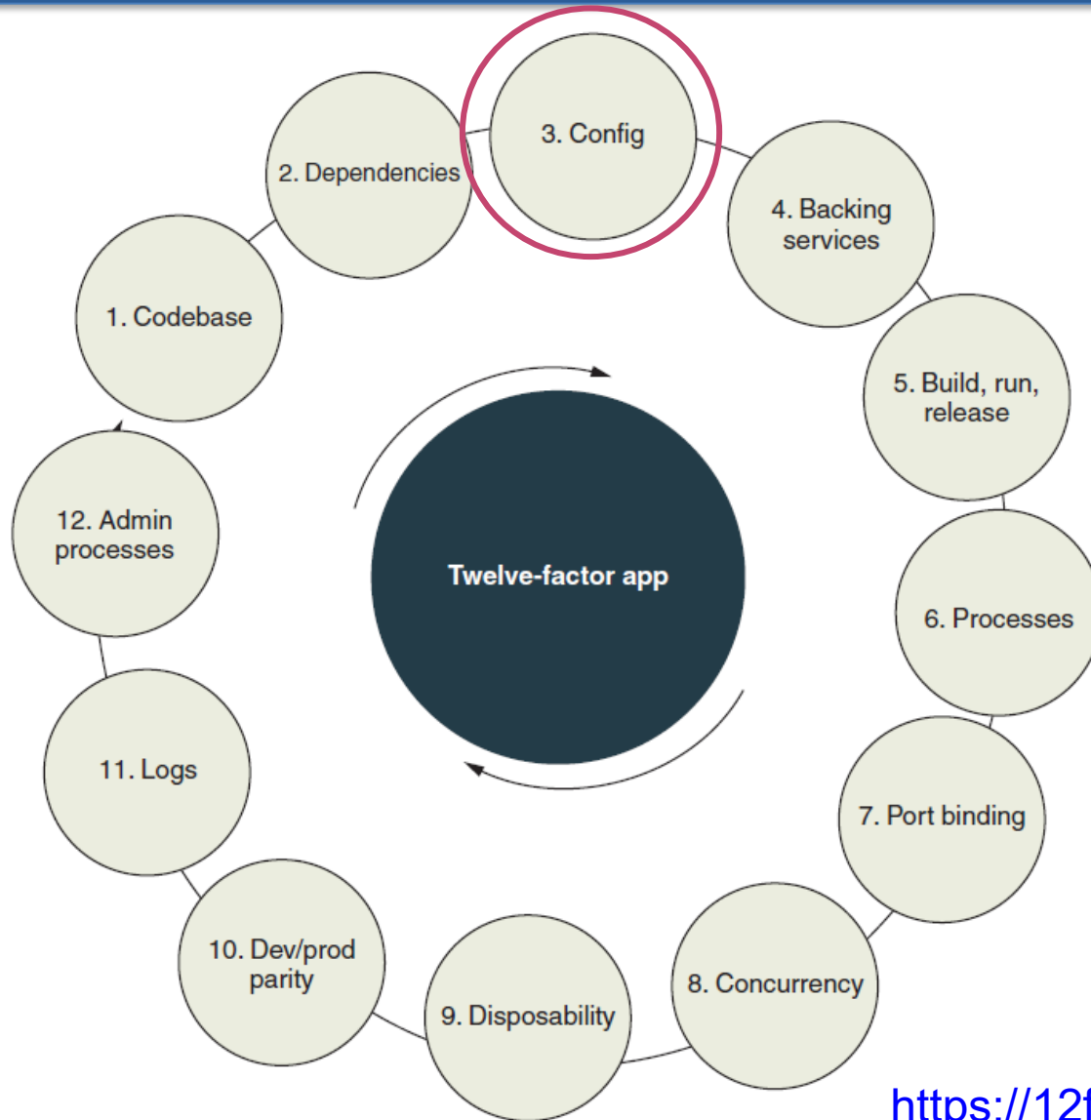
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

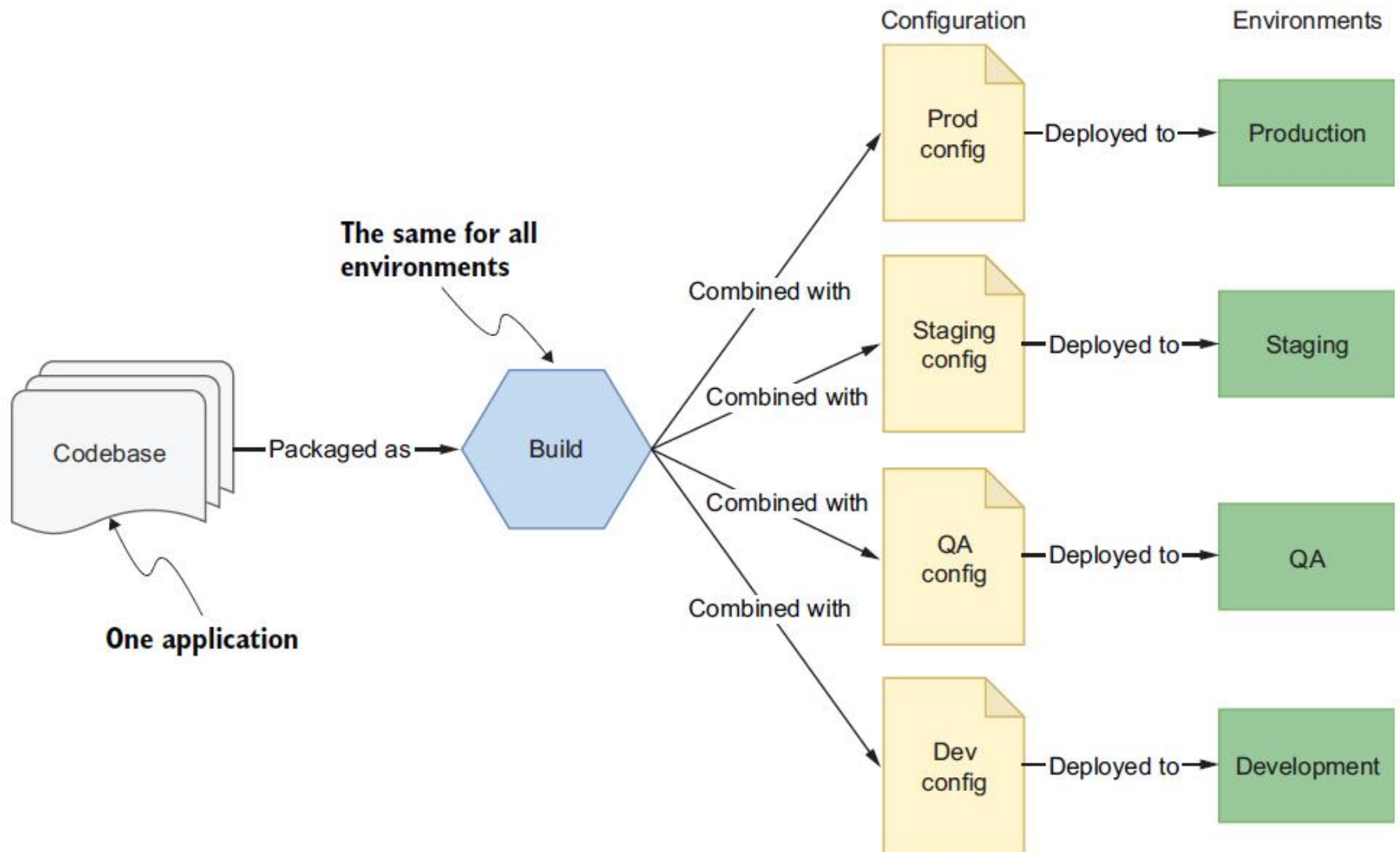
Lembram ?



Config

- Essa prática se refere a como você armazena as configurações da sua aplicação (especialmente as configurações específicas do ambiente).
- Nunca adicione configurações incorporadas ao seu código-fonte!
- Em vez disso, é melhor manter a sua configuração completamente separada do seu microserviço implantável.

Config



Configuração no Spring

- O termo "configuração" pode ter diferentes significados dependendo do contexto.
- Ao discutir os recursos principais do Spring Framework e o seu `ApplicationContext`, a configuração se refere aos beans (objetos Java registrados no Spring) que foram definidos para serem gerenciados pelo container do Spring e são injetados onde necessário.
- Por exemplo, é possível definir beans em um arquivo XML (configuração XML), em uma classe `@Configuration` (configuração Java) ou por meio de anotações como `@Component` (configuração baseada em anotações).

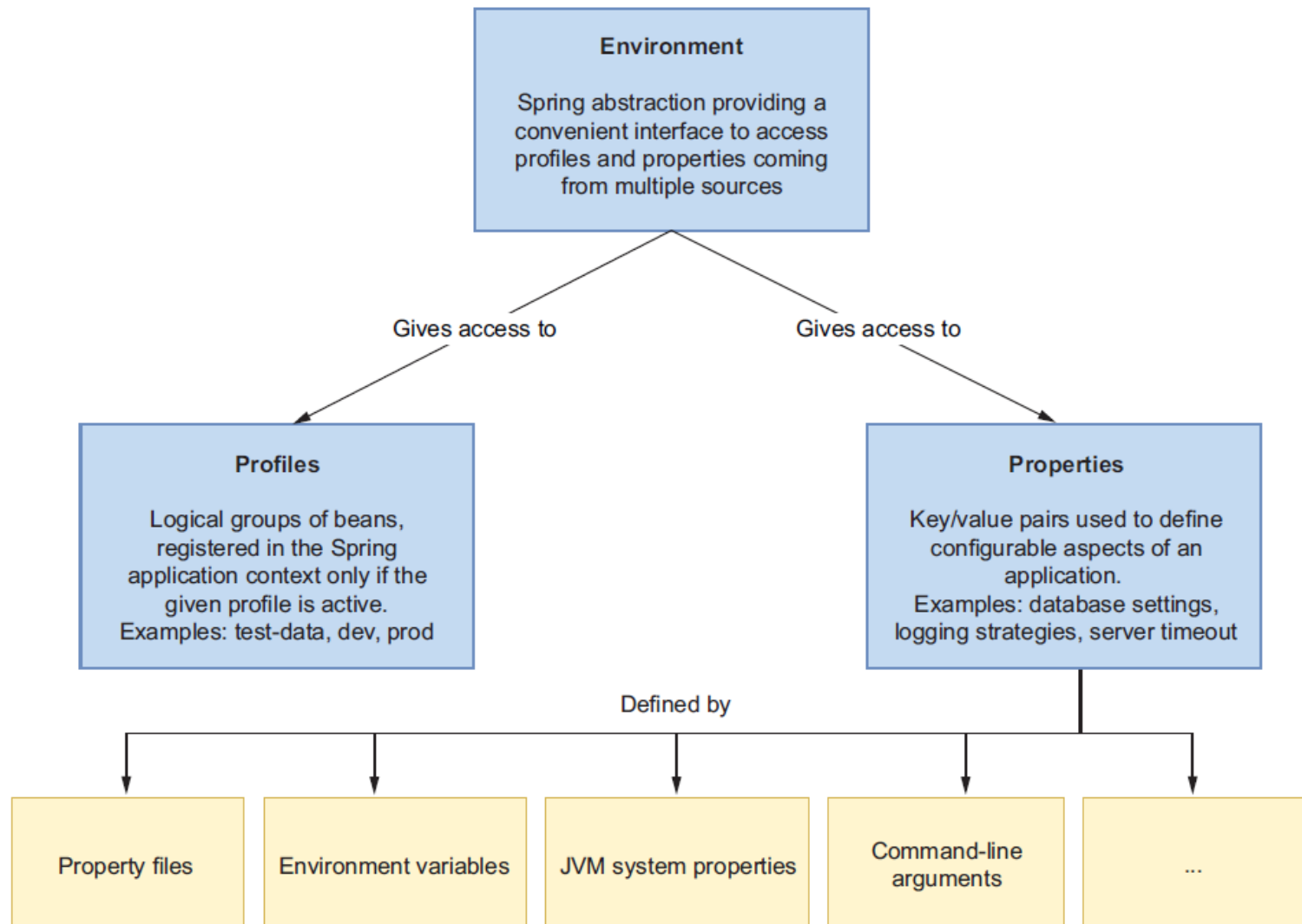
Configuração no Spring

- Desta forma, **a menos que especificado de outra forma**, sempre que menciono configuração, não me refiro ao conceito anterior, **mas sim a tudo que é provável de mudar entre implantações**, conforme definido pela metodologia dos 12 Fatores.

Configuração no Spring

Configuration strategy	Characteristics
Property files packaged with the application	■ These files can act as specifications of what configuration data the application supports. ■ These are useful for defining sensible default values, mainly oriented to the development environment.
Environment variables	■ Environment variables are supported by any operating system, so they are great for portability. ■ Most programming languages allow you access to the environment variables. In Java you can access them with the <code>System.getenv()</code> method. In Spring you can also rely on the <code>Environment</code> abstraction. ■ These are useful for defining configuration data that depends on the infrastructure and platform where the application is deployed, such as active profiles, hostnames, service names, and port numbers.
Configuration service	■ Provides configuration data persistence, auditing, and accountability. ■ Allows secrets management by using encryption or dedicated secret vaults. ■ This is useful for defining configuration data specific to the application, such as connection pools, credentials, feature flags, thread pools, and URLs to third-party services.

Configuração no Spring: Properties e Profiles



Configuração no Spring

- A seguir está uma lista priorizada de algumas das fontes de propriedades mais comuns, começando com a mais alta prioridade:
 1. Anotações `@TestPropertySource` em classes de teste
 2. Argumentos de linha de comando
 3. Propriedades do sistema JVM de `System.getProperties()`
 4. Variáveis de ambiente do sistema operacional de `System.getenv()`
 5. Arquivos de dados de configuração
 6. Anotações `@PropertySource` em classes `@Configuration`
 7. Propriedades padrão de `SpringApplication.setDefaultProperties`

Configuração no Spring

- **Quando a mesma propriedade é definida em várias fontes, aquela com a mais alta prioridade tem precedência.**
- Por exemplo, o que acontece se você especificar um valor para a propriedade `server.port` tanto em um arquivo de propriedades quanto em um argumento de linha de comando?
- Nesse caso, o valor especificado no argumento de linha de comando terá precedência sobre o valor definido no arquivo de propriedades. Isso ocorre porque o argumento de linha de comando é uma fonte de propriedade com uma prioridade mais alta do que o arquivo de propriedades. Portanto, o valor definido no argumento de linha de comando será o valor final usado para a propriedade `server.port`.

Configuração no Spring

- Os arquivos de dados de configuração também podem ser priorizados ainda mais, começando com a mais alta prioridade:
 1. Propriedades específicas do perfil em arquivos `application-{perfil}.properties` e `application-{perfil}.yml` empacotados fora do seu JAR
 2. Propriedades de aplicação em arquivos `application.properties` e `application.yml` empacotados fora do seu JAR
 3. Propriedades específicas do perfil em arquivos `application-{perfil}.properties` e `application-{perfil}.yml` empacotados dentro do seu JAR
 4. Propriedades de aplicação em arquivos `application.properties` e `application.yml` empacotados dentro do seu JAR
- Isso significa que, se houver um arquivo de propriedades específico do perfil fora do JAR, ele terá precedência sobre um arquivo de propriedades de aplicação fora do JAR. Da mesma forma, os arquivos de propriedades dentro do JAR podem ser substituídos por arquivos de propriedades específicos do perfil dentro do JAR, se existirem.

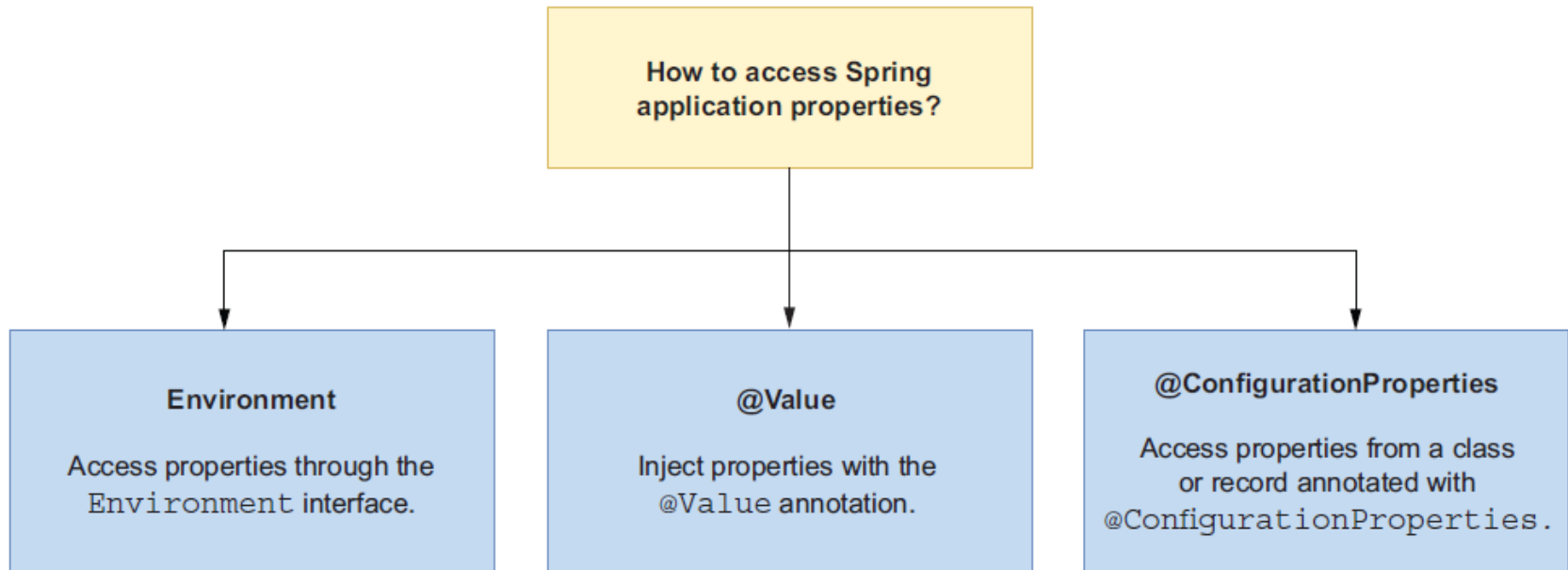
Formas de Configuração no Spring: Linha de Comando

```
$ java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar \
  --polar.greeting="Welcome to the catalog from CLI"
```

```
$ java -Dpolar.greeting="Welcome to the catalog from JVM" \
  -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar
```

```
$ POLAR_GREETING="Welcome to the catalog from ENV" \
  java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar
```

Configuração no Spring



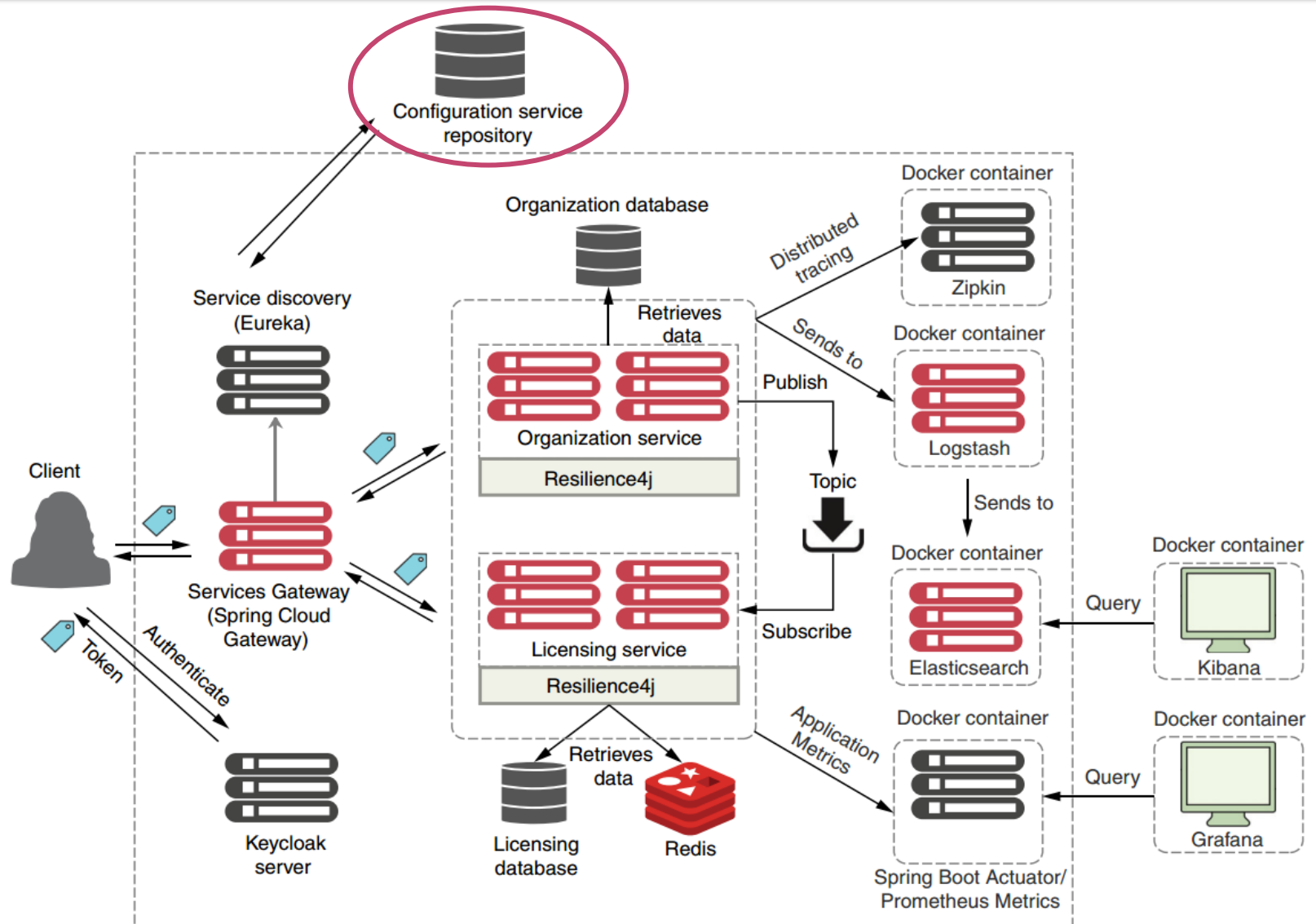
Usando ambiente para obter as propriedades da aplicação

```
@Autowired  
private Environment environment;  
  
public String getServerPort() {  
    return environment.getProperty("server.port");  
}
```

Usando ambiente para obter as propriedades da aplicação

```
@Value("${server.port}")  
private String serverPort;  
  
public String getServerPort() {  
    return serverPort;  
}
```

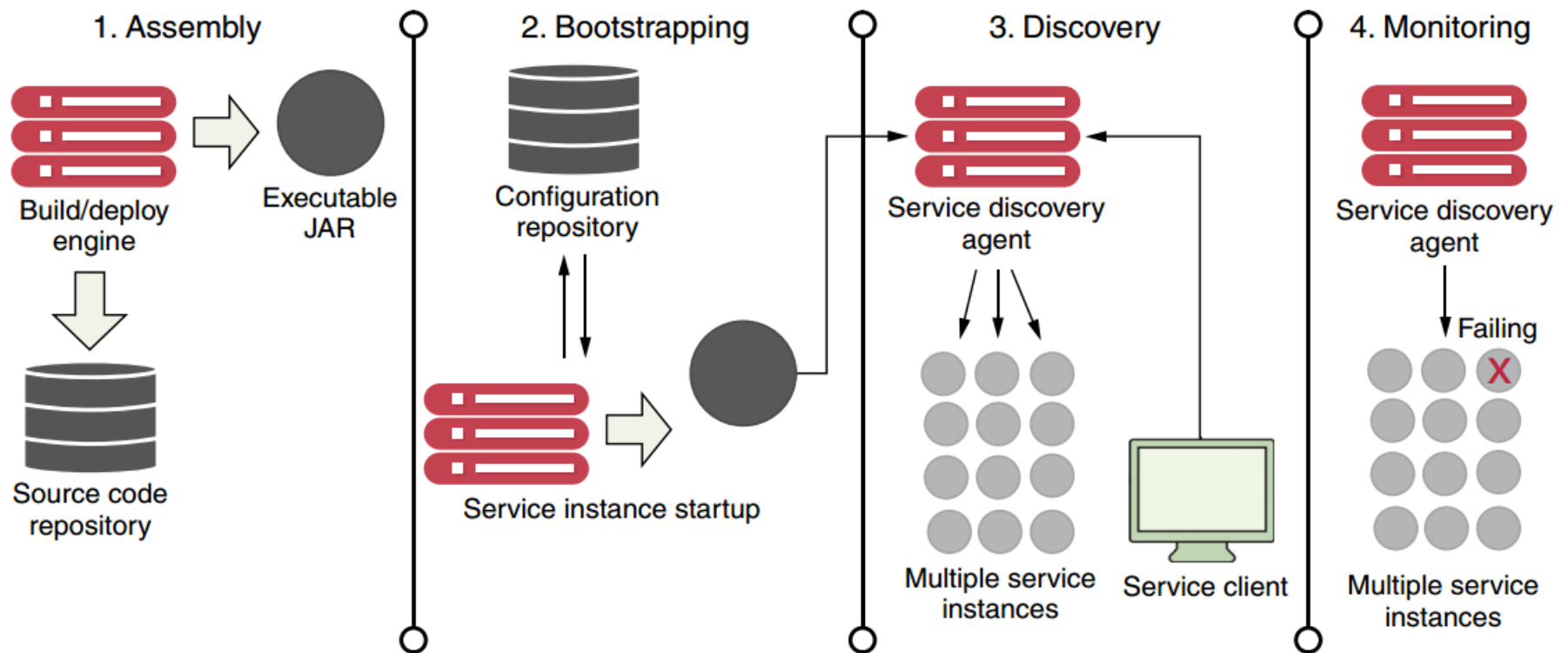

Configuração numa arquitetura Cloud Native



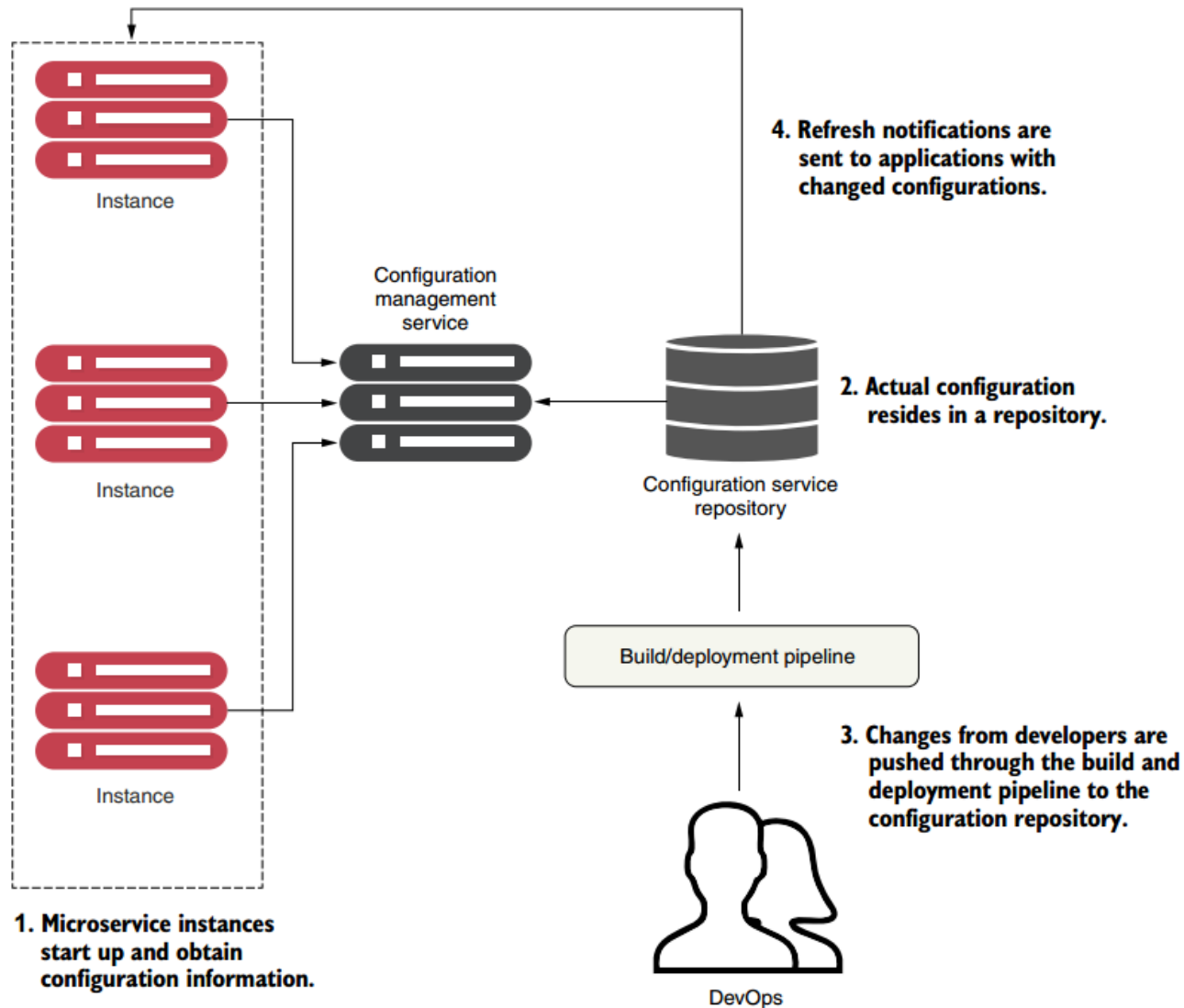
Por que eu deveria usar um Config Server?

- As variáveis de ambiente não oferecem **recursos de controle de acesso granulares**. Como você pode controlar o acesso aos dados de configuração?
- Os dados de configuração evoluirão e exigirão alterações, assim como o código da aplicação. Como você deve **acompanhar as revisões dos dados de configuração**? Como você deve auditar a configuração usada em uma versão?
- Depois de alterar seus dados de configuração, como você pode fazer com que **sua aplicação os leia em tempo de execução** sem exigir uma reinicialização completa?
- Quando o número de instâncias da aplicação aumenta, pode ser desafiador lidar com a **configuração de maneira distribuída** para cada instância. Como você pode superar esses desafios?
- Nem as propriedades do Spring Boot nem as variáveis de ambiente suportam criptografia de configuração, portanto, você não pode armazenar senhas com segurança. Como você deve gerenciar segredos?

Ciclo de Vida de um Microserviço



Operação do Serviço de Configuração



Opções de Serviços de Configuração

Project name	Description	Characteristics
etcd	Written in Go. Used for service discovery and key-value management. Uses the raft protocol (https://raft.github.io/) for its distributed computing model.	Very fast and scalable Distributable Command-line driven Easy to use and set up
Eureka	Written by Netflix. Extremely battle-tested. Used for both service discovery and key-value management.	Distributed key-value store Flexible but takes some effort to set up Offers dynamic client refresh out of the box
Consul	Written by HashiCorp. Similar to etcd and Eureka but uses a different algorithm for its distributed computing model.	Fast Offers native service discovery with the option to integrate directly with DNS Doesn't offer client dynamic refresh out of the box

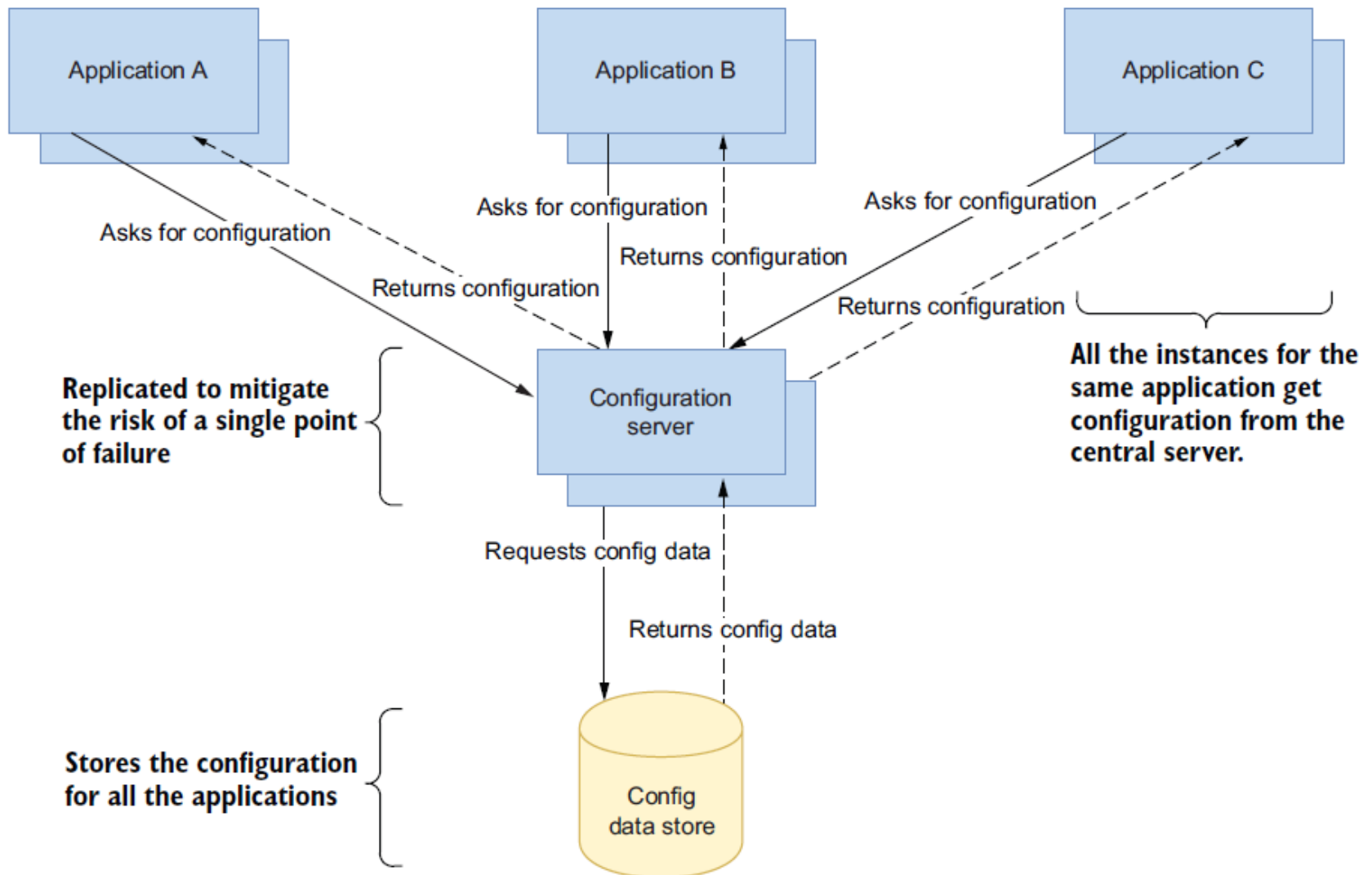
Opções de Serviços de Configuração

Project name	Description	Characteristics
Zookeeper	An Apache project. Offers distributed locking capabilities. Often used as a configuration management solution for accessing key-value data.	Oldest, most battle-tested of the solutions Most complex to use Can be used for configuration management, but consider only if you're already using Zookeeper in other pieces of your architecture
Spring Cloud Configuration Server	An open source project. Offers a general configuration management solution with different backends.	Non-distributed key-value store Offers tight integration for Spring and non-Spring services Can use multiple backends for storing configuration data including a shared filesystem, Eureka, Consul, or Git

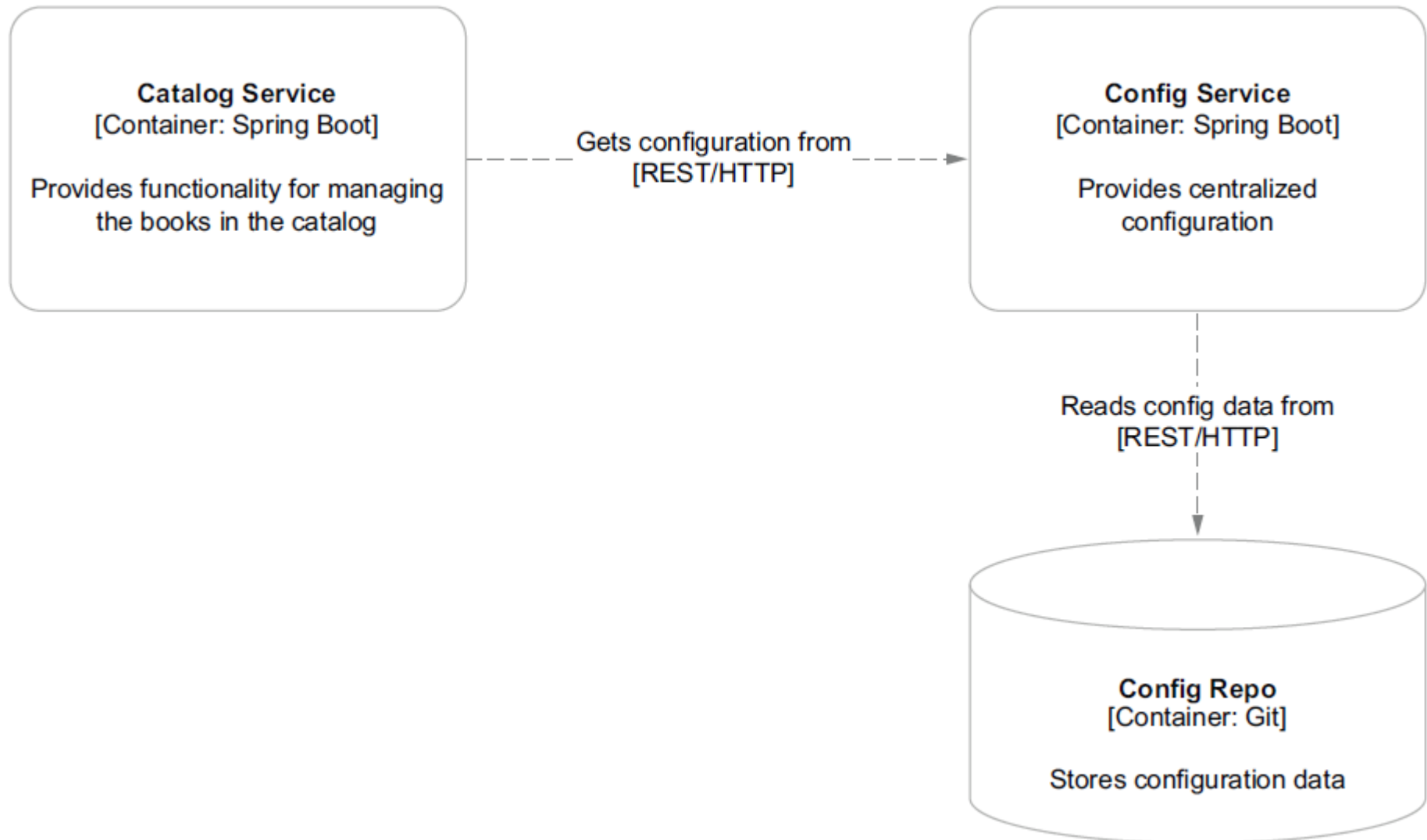
Spring Cloud Config

- O Spring Cloud Config fornece suporte tanto no lado do servidor quanto no lado do cliente para configurações externalizadas em um sistema distribuído.
- Com o Config Server, você tem um local central para gerenciar propriedades externas para aplicativos em todos os ambientes.
- Escolhemos essa solução porque:
 - O Spring Cloud Configuration Server é fácil de configurar e usar.
 - O Spring Cloud Config integra-se perfeitamente ao Spring Boot.
 - Você pode literalmente ler todos os dados de configuração do seu aplicativo com algumas anotações simples de usar.
 - O Config Server oferece várias opções de armazenamento de dados de configuração: pode-se integrar diretamente com a plataforma de controle de origem Git e com o HashiCorp Vault.

Exemplo de uso do Spring Cloud



Exemplo de uso do Spring Cloud



Criando Cloud Config Microservice

New Spring Starter Project 

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:



Criando Cloud Config Microservice

New Spring Starter Project Dependencies

Spring Boot Version: 2.5.0 (M3) ▾

Frequently Used:

☐ Config Client☐ Spring Boot Actuator☒ Spring Boot DevTools☐ Spring Web

Available:

config ✕

Alibaba

☐ Nacos Configuration

Developer Tools

☒ Spring Boot DevTools☐ Spring Configuration Processor

Google Cloud Platform

☐ GCP Support

Microsoft Azure

☐ Azure Support

Security

☐ Okta

Spring Cloud Config

☐ Config Client☒ Config Server

Selected:

X Spring Boot DevToolsX Config Server

Make DefaultClear Selection

?

< Back

Next >

Cancel

Finish

@EnableConfigServer

```
import org.springframework.boot.SpringApplication;
@EnableConfigServer
@SpringBootApplication
public class SpringCloudConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringCloudConfigServerApplication.class, args);
    }
}
```

@EnableConfigServer

- @EnableConfigServer permite criar um servidor de configuração que pode se comunicar com outras aplicações.
- O Config Server precisa saber qual repositório gerenciar.
- Existem várias opções, mas começa com um repositório baseado em sistema de arquivos Git.

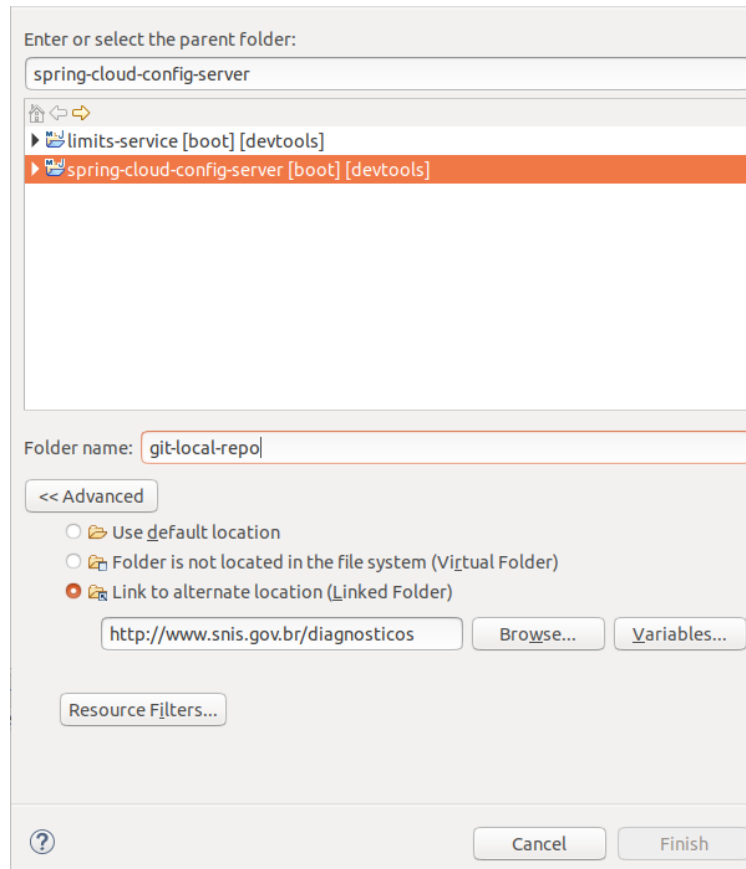
Integrar com o GIT

- Primeiro criar repositório local do GIT

```
nelio@Nelio-PC:~/Documentos/GoogleDrive$ mkdir git-local-repo
nelio@Nelio-PC:~/Documentos/GoogleDrive$ cd git-local-repo/
nelio@Nelio-PC:~/Documentos/GoogleDrive/git-local-repo$ ls
nelio@Nelio-PC:~/Documentos/GoogleDrive/git-local-repo$ git init
Initialized empty Git repository in /home/nelio/Documentos/GoogleDrive/git-local-repo/.git/
```

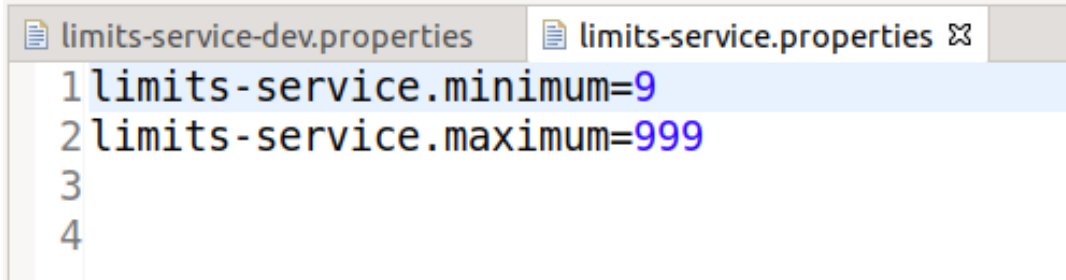
Integrar com o GIT

- Criar uma pasta no seu projeto associado ao GIT



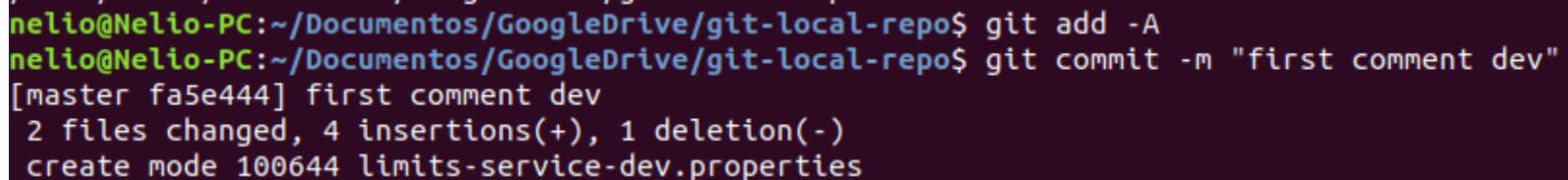
Integrar com o GIT

- Criar arquivos de configuração



```
limits-service-dev.properties limits-service.properties ⌵
1 limits-service.minimum=9
2 limits-service.maximum=999
3
4
```

- Subir as mudanças para o GIT



```
nelio@Nelio-PC:~/Documentos/GoogleDrive/git-local-repo$ git add -A
nelio@Nelio-PC:~/Documentos/GoogleDrive/git-local-repo$ git commit -m "first comment dev"
[master fa5e444] first comment dev
 2 files changed, 4 insertions(+), 1 deletion(-)
 create mode 100644 limits-service-dev.properties
```


Integrar com o GIT

- Dependendo das suas necessidades, você pode organizar a estrutura de pastas usando diferentes combinações, como estas:

```
{application}/application-{profile}.yaml  
{application}/application.yaml  
{application}-{profile}.yaml  
{application}.yaml  
application-{profile}.yaml  
application.yaml
```

Integrar com o GIT

master ▾

opdigitalconfig / config / opdigital-auth / + ▾

History

Find file

Web IDE

↓ ▾

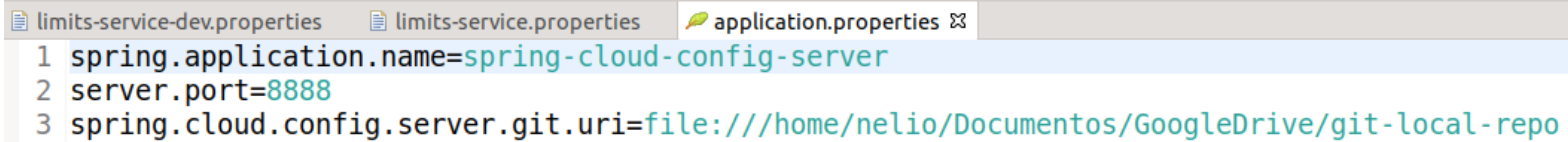
Clone ▾

8af537c8 

Name	Last commit	Last update
..		
 dev	Adicionado configfiles de opdigital-auth, ps: conferir a serverport pra ver se esta certa	11 months ago
 prod	Adicionado configfiles de opdigital-auth, ps: conferir a serverport pra ver se esta certa	11 months ago
 test	Adicionado configfiles de opdigital-auth, ps: conferir a serverport pra ver se esta certa	11 months ago

Integrar com o GIT

- Mudar application.properties indicando o Git repo.



The screenshot shows an IDE with three tabs: 'limits-service-dev.properties', 'limits-service.properties', and 'application.properties'. The 'application.properties' tab is active and contains the following content:

```
1 spring.application.name=spring-cloud-config-server
2 server.port=8888
3 spring.cloud.config.server.git.uri=file:///home/nelio/Documentos/GoogleDrive/git-local-repo
```

Outro Exemplo de Integrar com o GIT

- Este exemplo mostra como ter acesso a um GIT remoto. Mostra também como atribuir uma senha de acesso ao Config server

```
spring.application.name=opdigital-configserver  
server.port=8888  
spring.cloud.config.server.git.uri=https://projetos.imd.ufrn.br/xpto/xpto.git  
spring.cloud.config.server.git.search-paths='config/{application}/{profile}'
```

```
#Senha para ter acesso ao repositório do GIT  
spring.cloud.config.server.git.password=xpto123  
spring.cloud.config.server.git.username=xpto
```

```
#Senha para ter acesso ao ConfigServer  
spring.security.user.name=root  
spring.security.user.password=xpto
```

Acessar servidor de Configuração

- O Spring Cloud Config Server expõe propriedades por meio de uma série de endpoints usando diferentes combinações dos parâmetros {application}, {profile} e {label}:

```
/ {application} / {profile} [/ {label}]  
/ {application} - {profile} .yaml  
/ {label} / {application} - {profile} .yaml  
/ {application} - {profile} .properties  
/ {label} / {application} - {profile} .properties
```

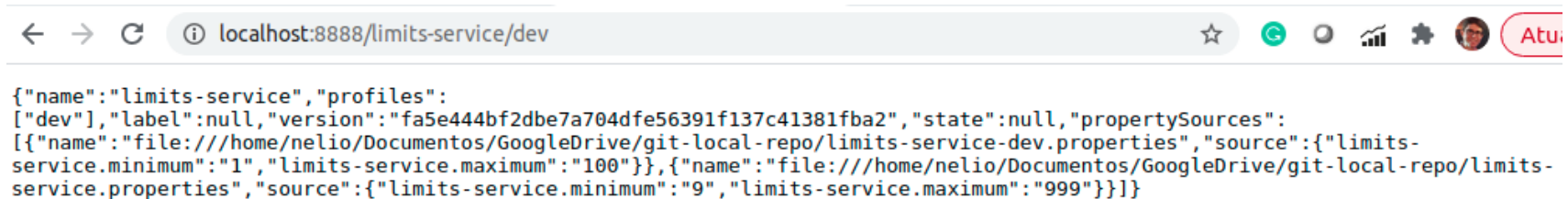
Acessar servidor de Configuração

- Exemplo de retorno:



Acessar servidor de Configuração

- Vamos testar no nosso exemplo:



The screenshot shows a web browser window with the address bar displaying 'localhost:8888/limits-service/dev'. The page content is a JSON object representing configuration data for a 'limits-service' in a 'dev' profile. The JSON includes a 'version' field, a 'state' field, and a 'propertySources' array with two entries defining minimum and maximum values for 'limits-service'.

```
{
  "name": "limits-service",
  "profiles": [
    {
      "dev": {
        "label": null,
        "version": "fa5e444bf2dbe7a704dfe56391f137c41381fba2",
        "state": null,
        "propertySources": [
          {
            "name": "file:///home/nelio/Documentos/GoogleDrive/git-local-repo/limits-service-dev.properties",
            "source": {
              "limits-service.minimum": "1",
              "limits-service.maximum": "100"
            }
          },
          {
            "name": "file:///home/nelio/Documentos/GoogleDrive/git-local-repo/limits-service.properties",
            "source": {
              "limits-service.minimum": "9",
              "limits-service.maximum": "999"
            }
          }
        ]
      }
    }
  ]
}
```

Integrar com o cliente

1. Spring profile, application name, and endpoint information passed to licensing service

```
Spring profile = dev  
Spring application name = licensing-service  
Spring Cloud config endpoint = http://localhost:8071
```



License service instance

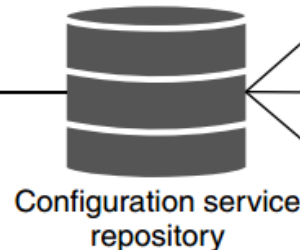
http://localhost:8071/
licensing-service/dev



Spring Cloud Config service

2. Licensing service contacts Spring Cloud Config service

3. Profile-specific configuration information retrieved from repository



Configuration service repository



licensing-service.properties



licensing-service-dev.properties



licensing-service-prod.properties

4. Property values passed back to licensing service

```
spring.jpa.hibernate.ddl-auto=none  
spring.jpa.database=POSTGRESQL  
spring.datasource.platform=postgres  
spring.jpa.show-sql = true  
spring.jpa.hibernate.naming-strategy =  
org.hibernate.cfg.ImprovedNamingStrategy  
spring.jpa.properties.hibernate.dialect =  
org.hibernate.dialect.PostgreSQLDialect  
spring.database.driverClassName= org.postgresql.Driver  
spring.datasource.testWhileIdle = true  
spring.datasource.validationQuery = SELECT 1  
spring.datasource.url = jdbc:postgresql://localhost:5432/ostock_dev  
spring.datasource.username = postgres  
spring.datasource.password = postgres  
...
```


Criando primeiro Microservice

New Spring Starter Project 

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

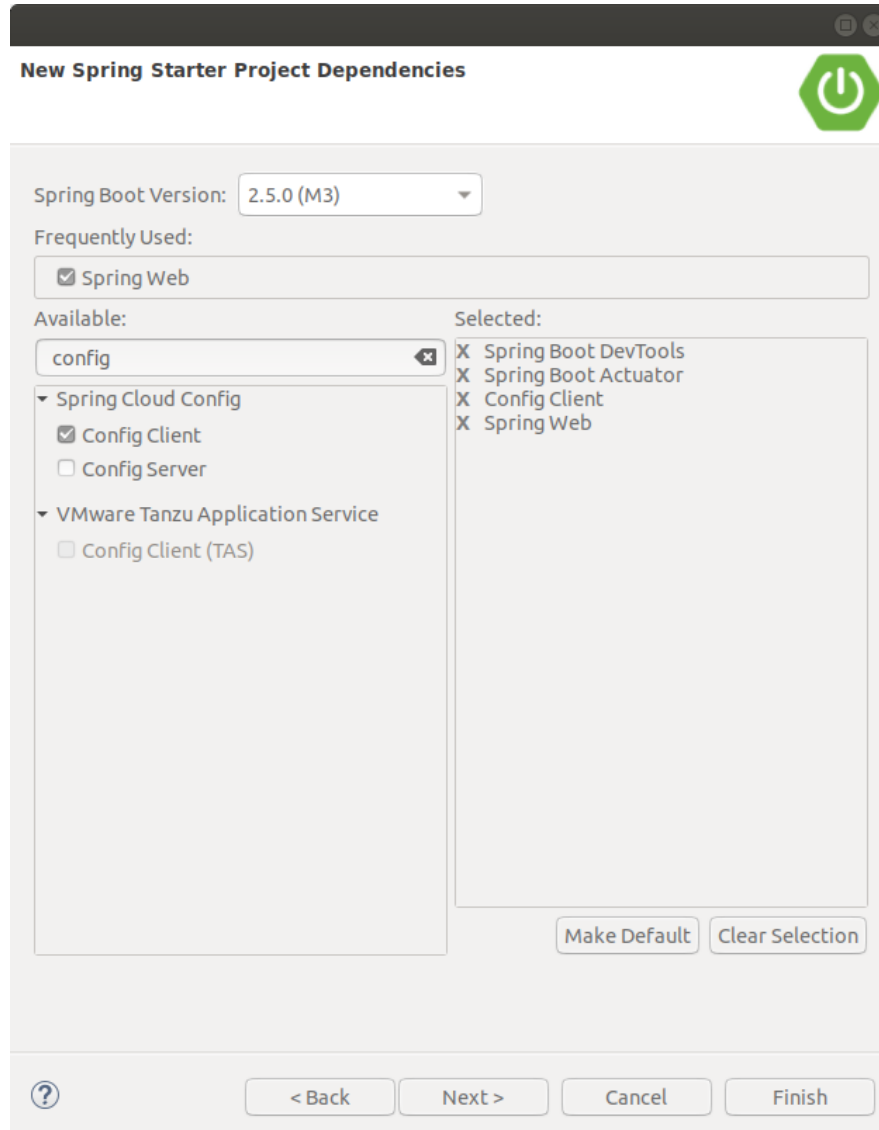
Package:

Working sets

☐ Add project to working sets

Working sets:

Criando primeiro Microservice



The image shows a 'New Spring Starter Project Dependencies' dialog box. At the top, it has a title bar with a green power button icon. Below the title bar, there's a 'Spring Boot Version' dropdown set to '2.5.0 (M3)'. Under 'Frequently Used', 'Spring Web' is checked. The 'Available' section on the left has a search bar with 'config' and a list of options: 'Spring Cloud Config' (expanded) with 'Config Client' checked and 'Config Server' unchecked, and 'VMware Tanzu Application Service' with 'Config Client (TAS)' unchecked. The 'Selected' section on the right lists 'Spring Boot DevTools', 'Spring Boot Actuator', 'Config Client', and 'Spring Web', each with an 'X' icon. At the bottom right of the dialog are 'Make Default' and 'Clear Selection' buttons. At the very bottom are navigation buttons: a help icon, '< Back', 'Next >', 'Cancel', and 'Finish'.

New Spring Starter Project Dependencies

Spring Boot Version: 2.5.0 (M3)

Frequently Used:

- ☒ Spring Web

Available:

config

- Spring Cloud Config
 - ☒ Config Client
 - ☐ Config Server
- VMware Tanzu Application Service
 - ☐ Config Client (TAS)

Selected:

- X Spring Boot DevTools
- X Spring Boot Actuator
- X Config Client
- X Spring Web

Make Default Clear Selection

? < Back Next > Cancel Finish

Criando primeiro Microservice

- ▼  limits-service [boot] [devtools]
 - ▼  src/main/java
 - ▼  ufrn.imd
 - ▶  Configuration.java
 - ▶  LimitsConfigurationController.java
 - ▶  LimitsServiceApplication.java
 - ▼  ufrn.imd.bean
 - ▶  LimitConfiguration.java

Criando Dto para os dados de Configuração

```
package ufrn.imd.bean;

public class LimitConfiguration {
    private int maximum;
    private int minimum;
    protected LimitConfiguration() {

    }
    public LimitConfiguration(int maximum, int minimum) {
        super();
        this.maximum = maximum;
        this.minimum = minimum;
    }
    public int getMaximum() {
        return maximum;
    }
    public int getMinimum() {
        return minimum;
    }
}
```

Criando uma classe para receber as configurações

```
@Component
@ConfigurationProperties("limits-service")
public class Configuration {

    private int minimum;
    private int maximum;

    public void setMinimum(int minimum) {
        this.minimum = minimum;
    }

    public void setMaximum(int maximum) {
        this.maximum = maximum;
    }

    public int getMinimum() {
        return minimum;
    }

    public int getMaximum() {
        return maximum;
    }
}
```

@ConfigurationProperties

- @ConfigurationProperties permite mapear propriedades com os arquivos Yaml inteiros para um objeto de forma fácil.

RestController

```
import org.springframework.beans.factory.annotation.Autowired;
@RestController
public class LimitsConfigurationController {

    @Autowired
    private Configuration configuration;

    public LimitsConfigurationController() {
        // TODO Auto-generated constructor stub
    }

    @GetMapping("/limits")
    public LimitConfiguration retrieveLimitsFromConfigurations() {
        LimitConfiguration limitConfiguration = new LimitConfiguration(configuration.getMaximum(),
            configuration.getMinimum());
        return limitConfiguration;
    }
}
```

Link entre Limits-service e Cloud Config

```
application.properties ✕  
1 server.port=8080  
2 spring.application.name=limits-service  
3  
4 spring.profiles.active=dev  
5 spring.config.import=optional:configserver:http://localhost:8888  
6
```


Subir Limits-service



localhost:8080/limits

```
{"maximum":100,"minimum":1}
```

Tornando seu cliente resiliente

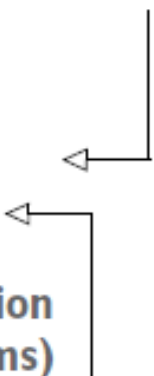
- Quando a integração com o servidor de configuração não é opcional, a aplicação falha ao iniciar se não conseguir entrar em contato com um servidor de configuração.
- Se o servidor estiver em funcionamento, ainda é possível enfrentar problemas devido à natureza distribuída da interação.
- Portanto, uma boa prática é definir alguns timeouts para fazer a aplicação falhar mais rapidamente.
 - Você pode usar a propriedade *spring.cloud.config.request-connect-timeout* para controlar o limite de tempo para estabelecer uma conexão com o servidor de configuração.
 - A propriedade *spring.cloud.config.request-read-timeout* permite limitar o tempo gasto lendo os dados de configuração do servidor.

Tornando seu cliente resiliente

- Exemplo:

```
spring:
  application:
    name: catalog-service
  config:
    import: "optional:configserver:"
  cloud:
    config:
      uri: http://localhost:8888
      request-connect-timeout: 5000
      request-read-timeout: 5000
```

Timeout on waiting
to connect to the
config server (ms)



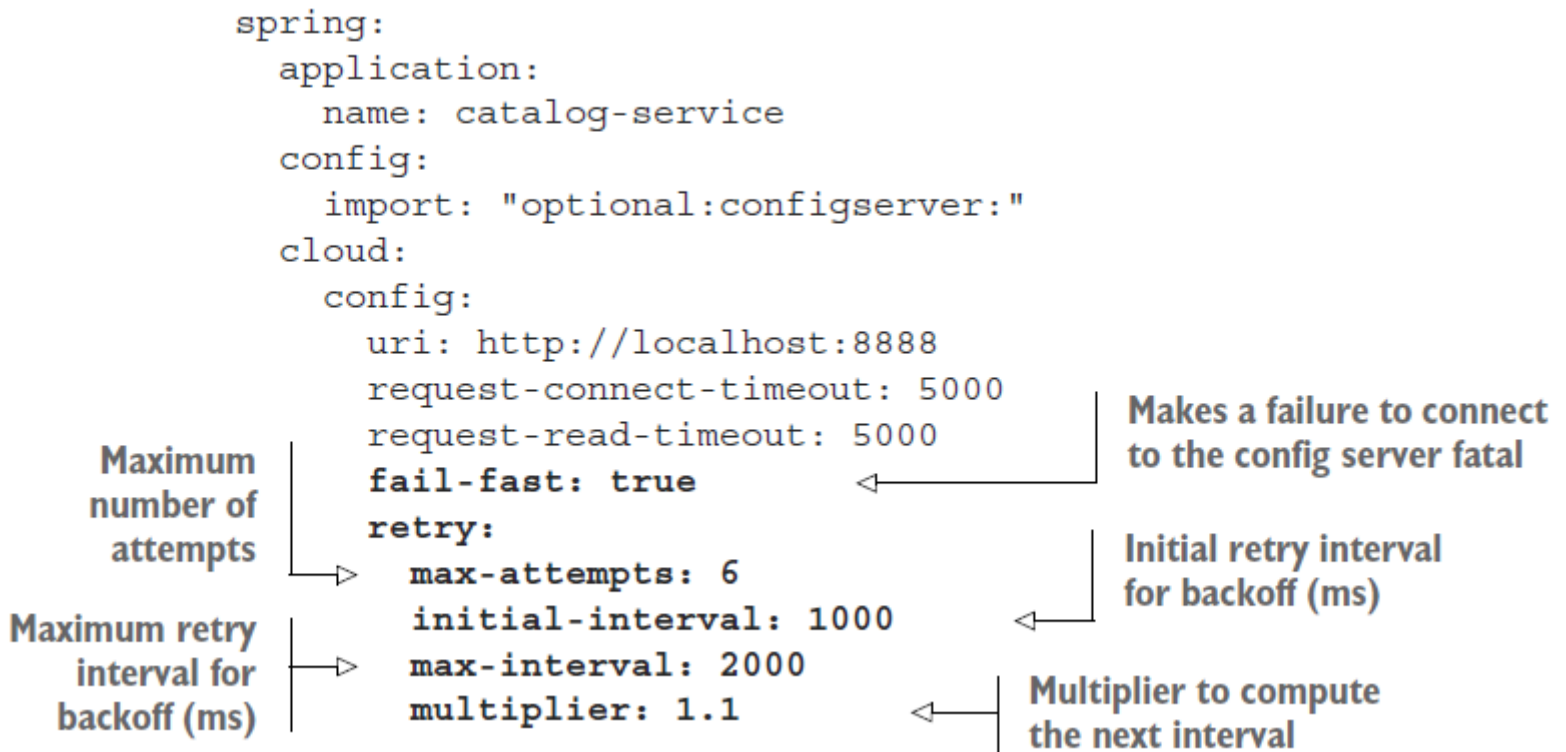
Timeout on waiting to read configuration
data from the config server (ms)

Tornando seu cliente resiliente

- Mesmo que o serviço de configuração esteja replicado, ainda há uma chance de que ele fique temporariamente indisponível quando uma aplicação cliente, como o serviço de catálogo, é iniciada.
- Nesse cenário, você pode aproveitar o padrão de **retry** e configurar a aplicação para tentar novamente se conectar com o servidor de configuração antes de desistir e falhar.
- A implementação de **retry** para o cliente Spring Cloud Config é baseada no *Spring Retry*.

Tornando seu cliente resiliente

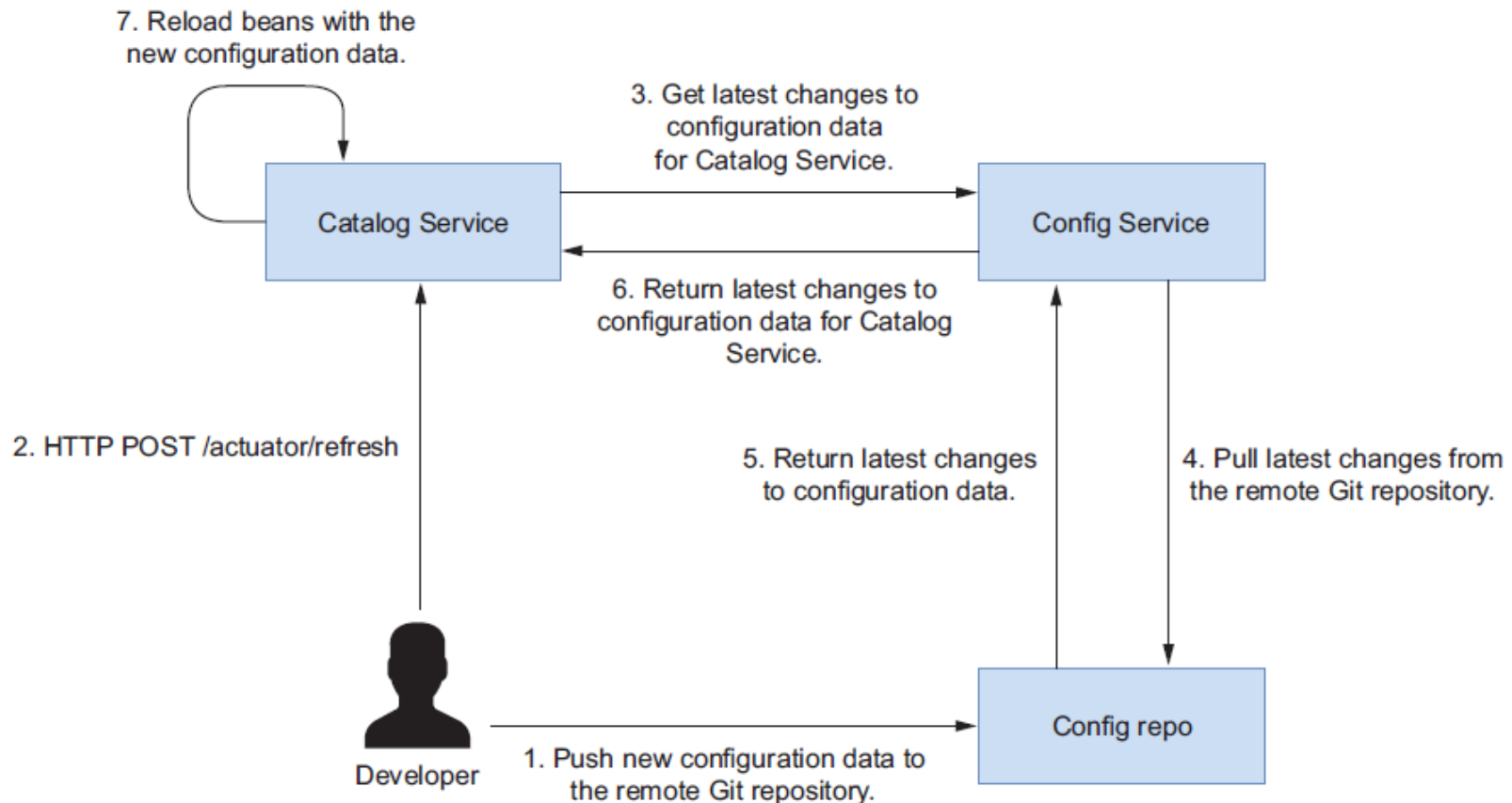
- O comportamento de retry é habilitado apenas quando a propriedade `spring.cloud.config.fail-fast` é definida como `true`.



Atualizando configuração em tempo de execução

- O que acontece quando novas alterações são enviadas para o repositório Git que está dando suporte ao Config Service?
- Em uma aplicação Spring Boot padrão, seria necessário reiniciá-la quando uma propriedade é alterada (seja em um arquivo de propriedades ou em uma variável de ambiente).
- No entanto, o Spring Cloud Config oferece a possibilidade de atualizar a configuração nas aplicações cliente em tempo de execução.
- Quando ocorrem novas alterações na configuração no repositório Git, o Spring Cloud Config tem a capacidade de notificar as aplicações cliente sobre essas mudanças. As aplicações cliente podem então solicitar uma atualização da configuração sem a necessidade de reiniciar completamente. Isso permite que as alterações na configuração sejam aplicadas em tempo real, sem interromper ou reiniciar as aplicações.

Atualizando configuração em tempo de execução



Atualizando configuração em tempo de execução

- Após realizar o *commit* e o *push* das novas alterações de configuração para o repositório Git remoto, você pode enviar uma solicitação POST para uma aplicação cliente através de um endpoint específico que irá disparar um evento *RefreshScopeRefreshedEvent* dentro do contexto da aplicação.
- Você pode contar com o projeto Spring Boot Actuator para expor o endpoint de atualização.

```
management:  
  endpoints:  
    web:  
      exposure:  
        include: refresh
```

Exposes the `/actuator/refresh`
endpoint through HTTP



Atualizando configuração em tempo de execução

- O evento de atualização, *RefreshScopeRefreshedEvent*, não terá efeito se não houver um componente ou bean ouvindo esse evento.
- Você pode usar a anotação *@RefreshScope* em qualquer bean que deseja recarregar sempre que uma atualização for acionada.
- A parte interessante é que, como você definiu suas propriedades personalizadas através de um bean *@ConfigurationProperties*, ele já está ouvindo automaticamente o evento *RefreshScopeRefreshedEvent*, portanto, você não precisa fazer nenhuma alteração em seu código.